

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ**

Ордена Трудового Красного Знамени

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«Московский технический университет связи и информатики»

Кафедра «Математическая Кибернетика и Информационные технологии»

Информационные технологии и программирование

Лабораторная работа №7

Многопоточность

Выполнил: Студент группы

БВТ2402

Юдин Владимир

Москва

2025

Цель работы

Изучить основы многопоточного программирования в Java, освоить работу с классами Thread, интерфейсом Runnable, а также механизмами асинхронного выполнения задач с использованием CompletableFuture. Получить практические навыки создания потоков, их синхронизации и организации параллельной обработки данных.

Задание 1. Многопоточная программа для вычисления суммы элементов массива

Создать два потока. Каждый поток должен вычислять сумму своей половины массива. Главный поток должен дождаться завершения вычислений обоих потоков и вывести итоговую сумму всех элементов массива.

Описание решения

1. Исходный массив разбивается на две равные части.
2. Для каждой части создаётся объект класса SumTask, реализующего интерфейс Runnable.
3. Два потока параллельно вычисляют частичные суммы.
4. Главный поток с помощью метода join() ожидает завершения потоков и суммирует результаты.

Используемые механизмы

- Thread
- Runnable
- Синхронизация через join()

```
● (base) vladimir@Noutbuk-Vladimir JAVA_LABS % cd /Users/vladimir/JAVA_LABS ; /usr/bin/javac SumTask.java  
-cp /Users/vladimir/Library/Application\ Support/Code/User/workspaceStorage/9fe48...  
Сумма элементов массива: 500500
```

```

class SumThread extends Thread {
    private int[] arr;
    private int start;
    private int end;
    private int sum;

    public SumThread(int[] arr, int start, int end) {
        this.arr = arr;
        this.start = start;
        this.end = end;
        this.sum = 0;
    }

    @Override
    public void run() {
        for (int i = start; i < end; i++) {
            sum += arr[i];
        }
    }

    public int getSum() {
        return sum;
    }
}

```

Задание 2. Многопоточная программа для поиска наибольшего элемента матрицы

Создать несколько потоков. Каждый поток должен обрабатывать одну строку матрицы и находить её максимальный элемент. После завершения всех потоков главный поток должен определить глобальный максимум.

Описание решения

1. Матрица представлена двумерным массивом `int[][]`.
2. Для каждой строки создаётся объект класса `MaxFinder`, реализующего `Runnable`.
3. Каждый поток находит локальный максимум своей строки.
4. Главный поток ждёт выполнения всех потоков, затем перебирает полученные значения и выбирает наибольшее.

Используемые механизмы

- `Thread`
- `Runnable`
- `join()` для синхронизации

- Передача строки как массива в поток

```
3 public class BiggestMatrix {
4     public static void main(String[] args) throws InterruptedException {
5         int[][] matrix = {
6             {1, 2, 3, 4},
7             {5, 6, 7, 8},
8             {9, 10, 11, 12},
9             {13, 14, 15, 16}
10        };
11
12        MaxFinder[] tasks = new MaxFinder[matrix.length];
13        Thread[] threads = new Thread[matrix.length];
14
15        for (int i = 0; i < matrix.length; i++) {
16            tasks[i] = new MaxFinder(matrix[i]);
17            threads[i] = new Thread(tasks[i]);
18            threads[i].start();
19        }
20
21        for (Thread t: threads) {
22            t.join();
23        }
24
25        int globalMax = Integer.MIN_VALUE;
26        for (MaxFinder task : tasks) {
27            if (task.getMax() > globalMax) {
28                globalMax = task.getMax();
29            }
30        }
31
32        System.out.println("Максимальный элемент в матрице: " + globalMax);
33    }
34 }
35
36 class MaxFinder implements Runnable {
37     private int[] array;
38     private int max = Integer.MIN_VALUE;
39
40     public MaxFinder(int[] array) {
41         this.array = array;
42         this.max = Integer.MIN_VALUE;
43     }
44
45     @Override
46     public void run() {
47         for (int num : array) {
48             if (num > max) {
49                 max = num;
50             }
51         }
52     }
53
54     public int getMax() {
55         return max;
56     }
57 }
```

```
84e6b09a07bb70f212af66d07577/redhat.java/jdt_ws/JAVA_LABS_efef6290/bin lab_no7.BiggestMatrix
Максимальный элемент в матрице: 16
```

Задание 3. Многопоточная модель склада с использованием `CompletableFuture`

Есть товары с разным весом. На складе работают три грузчика.

Они могут переносить товары одновременно, но суммарный вес переноса за одну итерацию не должен превышать 150 кг.

После набора партии грузчики «уезжают» переносить её и возвращаются.

Реализовать перенос товаров с использованием `CompletableFuture`.

Описание решения

1. Список товаров представлен как коллекция весов (`List<Integer>`).
2. Товары группируются в партии так, чтобы каждая партия не превышала 150 кг.
3. Создаётся пул потоков из трёх грузчиков (`ExecutorService`).
4. Для каждой партии запускается асинхронная задача с помощью `CompletableFuture.runAsync()`.
5. Все асинхронные задачи объединяются через `CompletableFuture.allOf()` для ожидания полного завершения.
6. Имитация переноса осуществляется задержкой `Thread.sleep()`.

Используемые механизмы

- `CompletableFuture`
- Фиксированный пул потоков (`Executors.newFixedThreadPool`)
- Асинхронное выполнение задач
- Объединение нескольких задач (`allOf`)

```

3  import java.util.*;
4  import java.util.Arrays;
5  import java.util.concurrent.*;
6
7  public class WarehouseTransfer {
8      private static final int MAX_WEIGHT = 150;
9      private static final int LOADERS = 3;
10
11      Run | Debug
12      public static void main(String[] args) {
13          List<Integer> goods = Arrays.asList(...a: 50, 70, 30, 90, 20, 60, 80, 40, 10, 100);
14
15          ExecutorService executor = Executors.newFixedThreadPool(LOADERS);
16          List<List<Integer>> batches = createBatches(goods);
17
18          System.out.println("Сформировано партий для переноса: " + batches.size());
19
20          List<CompletableFuture<Void>> futures = new ArrayList<>();
21
22          for (List<Integer> batch: batches) {
23              CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
24                  int sum = batch.stream().mapToInt(Integer::intValue).sum();
25                  System.out.println("Перенос партии " + batch + " с общим весом " + sum);
26                  simulateTransferDelay();
27                  System.out.println("Партия " + batch + " успешно перенесена.");
28              }, executor);
29
30              futures.add(future);
31          }
32
33          CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
34          executor.shutdown();
35          System.out.println("Все партии успешно перенесены.");
36      }
37  }

```

```

Code/User/workspaceStorage/9fe484e6b09a07bb70f212af66d07577/redhat.java/jdt_ws/JAVA_LABS_efef6290/bin
Сформировано партий для переноса: 4
Перенос партии [90, 20] с общим весом 110
Перенос партии [50, 70, 30] с общим весом 150
Перенос партии [60, 80] с общим весом 140
Партия [50, 70, 30] успешно перенесена.
Партия [60, 80] успешно перенесена.
Партия [90, 20] успешно перенесена.
Перенос партии [40, 10, 100] с общим весом 150
Партия [40, 10, 100] успешно перенесена.
Все партии успешно перенесены.
(base) vladimir@Noutbuk-Vladimir JAVA_LABS %

```

Выводы

В ходе выполнения лабораторной работы были изучены основные средства многопоточного программирования в Java.

Были рассмотрены два подхода:

1. Классическая модель потоков (Thread, Runnable).

Позволяет вручную создавать и контролировать выполнение потоков.

Использована для параллельного суммирования массива и поиска максимума в матрице.

2. Асинхронная модель (CompletableFuture).

Предоставляет более гибкий и современный механизм управления асинхронными задачами.

Позволяет легко комбинировать операции, не блокируя основной поток.

Использована для моделирования работы склада и распределения нагрузки между грузчиками.